Exploratory Data Analysis and Visualization

Assignment II

Customer Churn Analysis in the Telecom Sector: Exploratory Data Analysis, Visualization and Deep Learning

- **Project Title:** Customer Churn Analysis in the Telecom Sector: Exploratory Data Analysis, Visualization and Deep Learning
- **Course:** U21ADP05 - Exploratory Data Analysis and Visualization (Assignment II)
- **Student Name:** SRI SIVARANJANI M
- **Roll No.:** 24AD115
- **Class :** AD - B
- **Department:** Artificial Intelligence and Data Science
- **Institution:** KPR Institute of Engineering and Technology
- **Year / Semester / Batch:** III Year / V Semester / 2024-2028
- **Course Coordinator:** Ms.Revathi
- **Date of Submission: 14/10/2026**
- **GitHub Repository:** https://github.com/SrisivaranjaniM/telco-churn-eda-project

2. Abstract

Customer churn is a major source of lost revenue for telecom providers. This project applies exploratory data analysis and visualization to the IBM Telco Customer Churn dataset (7,043 customers, 21 features) to identify the factors associated with customers leaving. Data cleaning handled blank TotalCharges values, followed by categorical encoding, standardization and a stratified 70/15/15 split. Seven visualizations and statistical tests (chi-square, t-test, distribution fitting) explored univariate, bivariate and distribution patterns. A Multi-Layer Perceptron was trained with class weighting, early stopping and a basic hyperparameter search. The final model (128 units, dropout 0.4) achieved 74.6% test accuracy, 80% churn recall and an AUC of 0.839. Tenure, monthly charges and contract type were the most influential factors, suggesting that retention efforts should focus on new, month-to-month customers.

3. Introduction and Objective

3.1 Background. Telecommunication is a highly competitive sector, and acquiring a new customer is generally far more expensive than retaining an existing one. Customer churn, meaning customers discontinuing the service, therefore directly affects profitability. Being able to identify at-risk customers early allows a company to offer targeted incentives, better contracts or improved support.

3.2 Problem statement. Given customer demographics, subscribed services and billing information, identify the patterns that distinguish customers who churn from those who stay, and build a model that predicts churn.

3.3 Objectives.

1. Select and describe a real-world dataset with at least 15 features.
2. Perform EDA and preprocessing: missing values, outliers, duplicates, encoding, scaling and train/validation/test splitting.
3. Create at least five meaningful visualizations covering all seven stages of visualization, with interpretation.
4. Carry out univariate, distribution-based and bivariate statistical analysis.
5. Build a deep learning model (MLP) with basic hyperparameter tuning and feature importance.
6. Evaluate the model using loss and accuracy curves, a confusion matrix and an ROC curve with AUC.
7. Derive practical conclusions and recommendations.

3.4 Tools used. Python 3, Pandas, NumPy, Matplotlib, Seaborn [4], SciPy, scikit-learn [6], TensorFlow/Keras [5], Google Colab, GitHub.

4. Dataset Description

- **Source:** IBM sample dataset "Telco Customer Churn", downloaded from Kaggle: https://www.kaggle.com/datasets/blastchar/telco-customer-churn
- **Size:** 7,043 records and 21 columns (about 1 MB, CSV).
- **Target variable:** Churn (Yes/No), indicating whether the customer left within the last month.
- **Class balance:** about 26.5% of customers churned and 73.5% stayed (5,174 / 1,869).
- **Basic statistics:** tenure has a mean of 32.37 months (std 24.56, range 0 to 72) and MonthlyCharges has a mean of 64.76 (std 30.09, range 18.25 to 118.75).

Table 1: Description of fields

#	Field	Type	Description
1	customerID	Identifier	Unique customer ID (dropped, no predictive value)
2	gender	Categorical	Male / Female
3	SeniorCitizen	Binary	1 if the customer is a senior citizen
4	Partner	Categorical	Has a partner (Yes/No)
5	Dependents	Categorical	Has dependents (Yes/No)
6	tenure	Numerical	Months with the company
7	PhoneService	Categorical	Has phone service
8	MultipleLines	Categorical	Has multiple phone lines
9	InternetService	Categorical	DSL / Fiber optic / No
10	OnlineSecurity	Categorical	Online security add-on
11	OnlineBackup	Categorical	Online backup add-on
12	DeviceProtection	Categorical	Device protection add-on
13	TechSupport	Categorical	Tech support add-on
14	StreamingTV	Categorical	Streams TV
15	StreamingMovies	Categorical	Streams movies
16	Contract	Categorical	Month-to-month / One year / Two year
17	PaperlessBilling	Categorical	Paperless billing (Yes/No)
18	PaymentMethod	Categorical	Electronic check / Mailed check / Bank transfer / Credit card
19	MonthlyCharges	Numerical	Amount charged per

#	Field	Type	Description
			month
20	TotalCharges	Numerical	Total amount charged to date
21	Churn	Target	Yes / No

Why this dataset suits the problem: it contains demographic, service and billing features, is real-world in nature, exceeds the 15-feature requirement, and has a clear business use (customer retention).

5. EDA and Preprocessing

5.1 Data understanding. The dataset was loaded with Pandas. The functions info(), describe(), isnull().sum() and value_counts() were used to inspect data types, summary statistics, missing values and class balance.

5.2 Missing values. TotalCharges was stored as text because 11 records contained blank strings. These belonged to customers with tenure = 0 (new customers not yet billed). The column was converted to numeric using pd.to_numeric(errors="coerce") and the missing entries were filled with the median, which is robust to skewness.

5.3 Duplicates. No duplicate records were found, so none were removed.

5.4 Outliers. The IQR method (values beyond 1.5 x IQR from Q1/Q3) was applied to tenure, MonthlyCharges and TotalCharges. 0 outliers were found in tenure, 0 outliers in MonthlyCharges and 0 outliers in TotalCharges. No significant outliers were detected.

5.5 Data cleaning and transformation. The customerID column was dropped. The labels "No internet service" and "No phone service" were merged into "No" because they carry the same meaning as "No" for the add-on feature.

5.6 Encoding. Yes/No columns and gender were mapped to 0/1. Multi-class columns (InternetService, Contract, PaymentMethod) were one-hot encoded with drop_first=True to avoid the dummy variable trap.

5.7 Scaling. tenure, MonthlyCharges and TotalCharges were standardized using StandardScaler (zero mean, unit variance). The scaler was fitted on the training set only and applied to validation and test sets, to prevent data leakage.

5.8 Splitting. The data was split into 70% training, 15% validation and 15% test sets with stratification on Churn, so that all three sets keep the same churn ratio. The Sizes - Training: (4930,23) , Validation: (1056,23) , Test: (1057,23)

5.9 Key insights from EDA.

- The data is imbalanced (about one churner for every three non-churners), so accuracy alone is misleading and class weighting is used.
- Missing data was minimal and confined to new customers.
- Average customer tenure was 32.37 months, with a standard deviation of 24.56 months.
- Average monthly charges were 64.76 with a standard deviation of 30.09

6. Data Visualization

The seven stages of visualization (Ben Fry) [3] were followed:

Stage	Application in this project
1. Acquire	Dataset downloaded from Kaggle
2. Parse	CSV loaded; TotalCharges converted from text to numeric; fields structured by type
3. Filter	Removed duplicates and customerID; handled missing values
4. Mine	Descriptive statistics, correlation, chi-square test, t-test, distribution fitting
5. Represent	Selected chart types suited to each data type (bar, histogram, box, scatter, heatmap, pairplot)
6. Refine	Added titles, axis labels, legends, consistent colors and 150 dpi resolution
7. Interact	Static charts were produced, but they can be made interactive in a dashboard (Tableau, Power BI or Plotly), where users could filter by Contract type, tenure range or internet service and hover to see churn counts.

Figure 1: Churn distribution (countplot)

- **What it shows:** the number of customers who stayed versus churned.
- **Why created:** to check class balance before modelling.
- **Insight:** The dataset contains 5174 non-churners (73.46%) and 1869 churners(26.54%), indicating an imbalanced dataset.

Figure 1

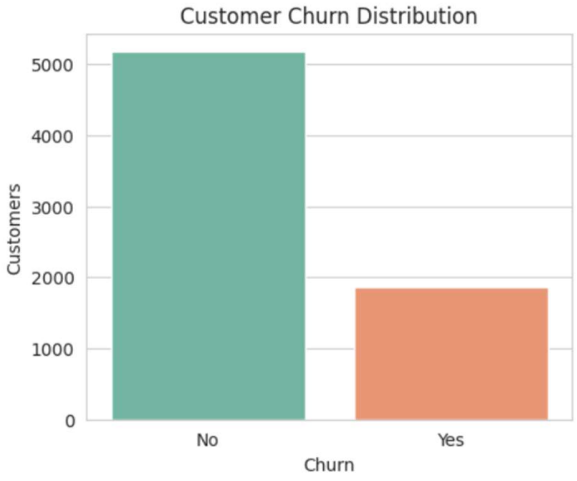


Figure 2: Tenure distribution by churn (histogram with KDE)

- **What it shows:** how many months customers have stayed, split by churn status.
- **Why created:** to see whether loyalty length relates to leaving.
- **Insight:** Churners are concentrated among customers with shorter tenure, whereas non-churners include many long-tenure customers. This suggests that customer retention improves with longer relationships.

Figure 2

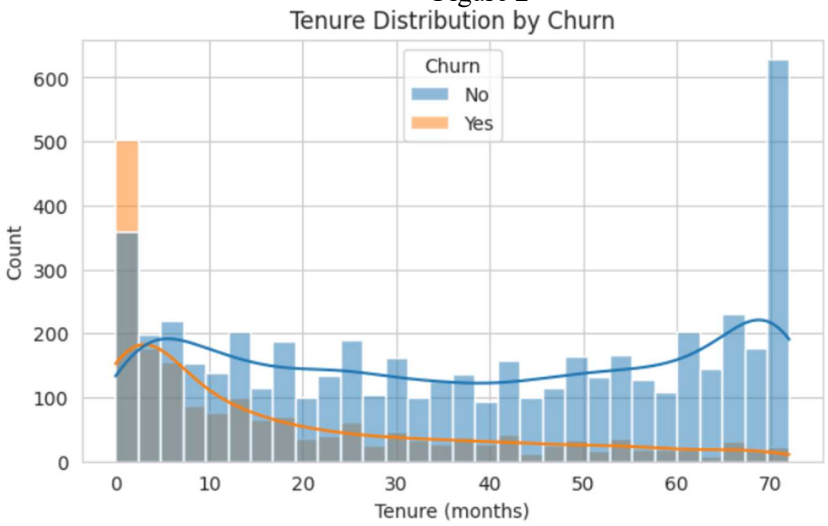


Figure 3: Monthly charges vs churn (boxplot)

- **What it shows:** the median, spread and outliers of monthly bills for each group.
- **Why created:** to test whether higher prices are linked to churn.
- **Insight:** Churners have a higher median monthly charge, approximately 79.650, compared with approximately 64.425 for non-churners.

Figure 3

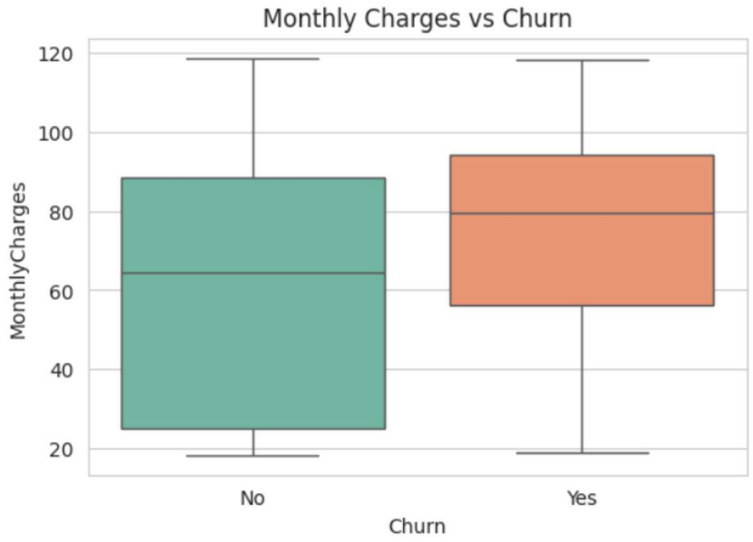


Figure 4: Churn by contract type (grouped bar chart)

- **What it shows:** churn counts for month-to-month, one-year and two-year contracts.
- **Why created:** contract commitment is a likely driver of retention.
- **Insight:** Month-to-month contracts have substantially more churn than one-year and two-year contracts. Approximate churn rates are 42.7% for month-to-month, 11.3% for one-year and 2.8% for two-year contracts.

Figure 4

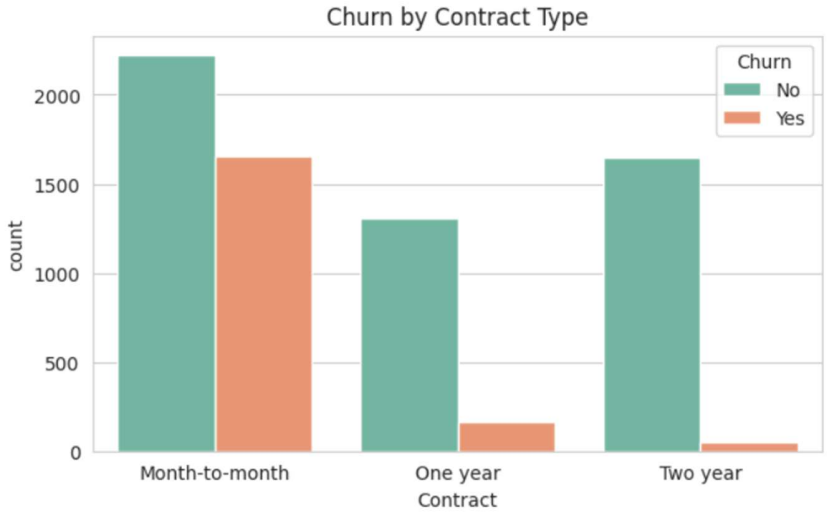


Figure 5: Tenure vs monthly charges (scatterplot colored by churn)

- **What it shows:** the relationship between tenure and monthly charges, with churn status as color.
- **Why created:** to see whether short-tenure, high-bill customers form a risk cluster.
- **Insight:** Churners are more concentrated among customers with short tenure and medium-to-high monthly charges. Long-tenure customers are more commonly non-churners.

Figure 5

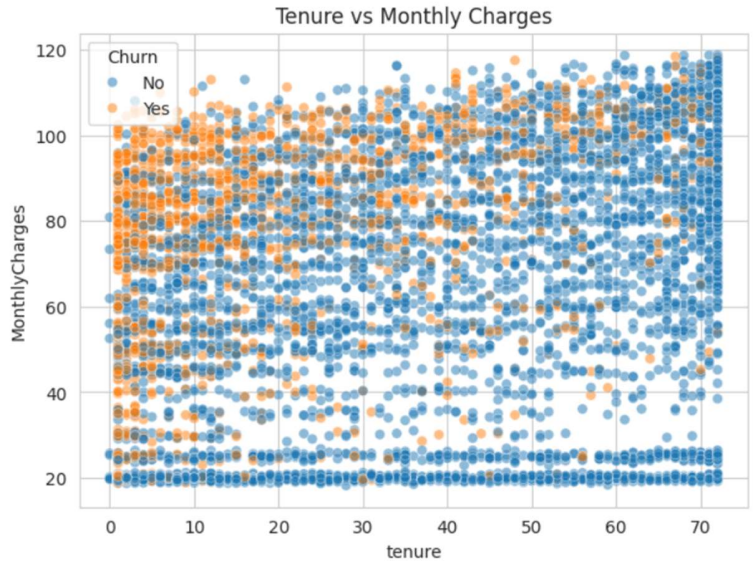


Figure 6: Correlation heatmap

- **What it shows:** Pearson correlation among numeric variables.
- **Why created:** to detect relationships and multicollinearity.
- **Insight:** The correlation between tenure and TotalCharges is approximately 0.83, between tenure and MonthlyCharges is 0.25, and between MonthlyCharges and TotalCharges is 0.65. Tenure and TotalCharges therefore have a strong positive relationship

Figure 6

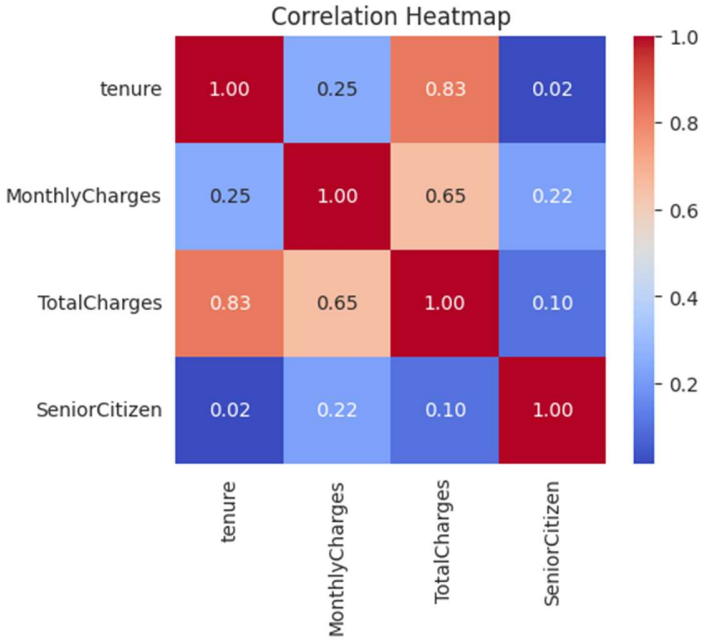
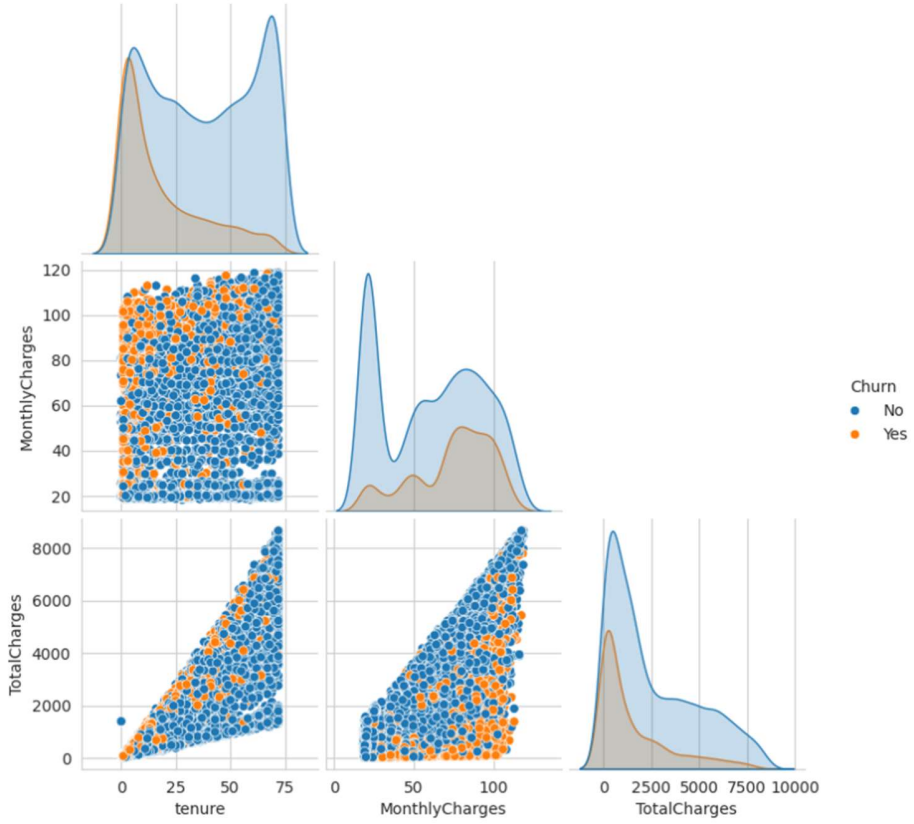


Figure 7: Pairplot

- **What it shows:** pairwise scatterplots and distributions of tenure, MonthlyCharges and TotalCharges colored by churn.
- **Why created:** to view all pairwise relationships in one figure.
- **Insight:** The pairplot shows a strong positive relationship between tenure and TotalCharges. Churn is more clearly separated along tenure than along monthly charges alone.

Figure 7



6.1 Statistical and distribution-based analysis (CO4)

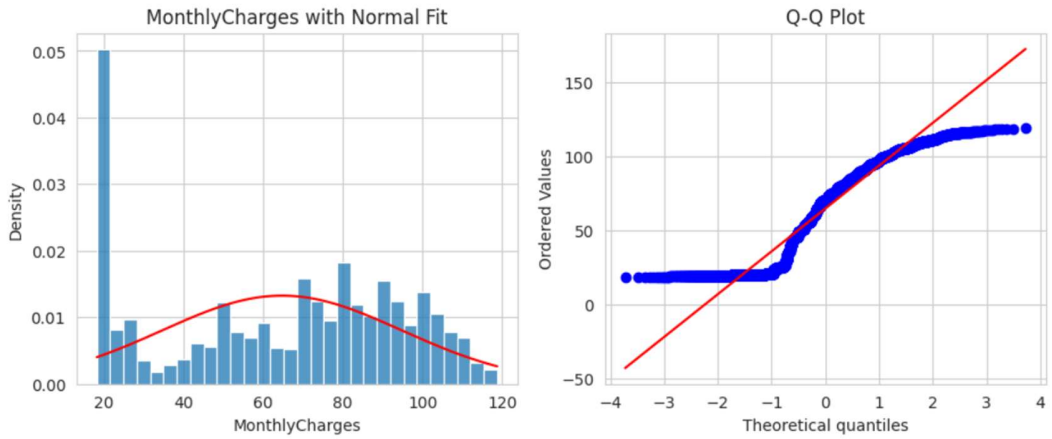
Univariate analysis. For MonthlyCharges: mean 64.7617, median was 70.35, standard deviation was 30.0900, skewness -0.22047, kurtosis -1.25721. The mean is lower than the median, and the skewness value of -0.2205 indicates a slightly negatively skewed distribution. The negative kurtosis value indicates a flatter distribution compared with a normal distribution.

	tenure	MonthlyCharges	TotalCharges
mean	32.371149	64.761692	2281.916928
median	29.000000	70.350000	1397.475000
std	24.559481	30.090047	2265.270398
skew	0.239540	-0.220524	0.963789
kurt	-1.387372	-1.257260	-0.226400

Continuous distribution. Figure 8 shows the MonthlyCharges histogram with a fitted normal curve and a Q-Q plot. The Shapiro–Wilk normality test produced a statistic of **0.9253** and a p-value of $\mathbf{4.5773 \times 10^{-15}}$. Since the p-value is less than 0.05, the null hypothesis of normality is rejected. Therefore, the MonthlyCharges distribution significantly deviates from a normal distribution.

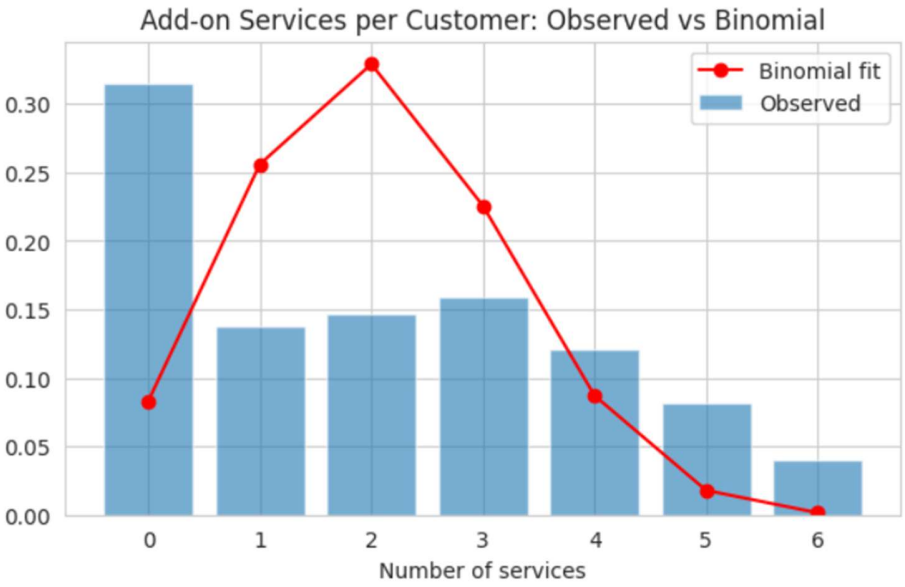
The histogram shows a tall spike near 20 and a broad second group between about 70 and 100, which is very different from the single bell shape of the fitted normal curve. In the Q-Q plot the points are flat at the low end and bend away from the red line in both tails, confirming the departure from normality

Figure 8



Discrete distribution. Figure 9 compares the number of add-on services per customer (0 to 6) with a Binomial(n = 6, p = 0.339651) distribution. Differences between the observed frequencies and the fitted Binomial distribution suggest that the distribution of add-on services may not perfectly follow the Binomial model. This may indicate that service selections are not independent, as customers who subscribe to one add-on service may be more likely to subscribe to others. About 31% of customers have no add-on service at all, whereas the Binomial model predicts about 8%, and the observed shares at 4 to 6 add-ons are higher than predicted. The Binomial model therefore fits poorly.

Figure 9



Bivariate analysis.

- Chi-square test of Contract vs Churn: $p = 5.863 \times 10^{-258}$. Since $p < 0.05$, contract type and churn are significantly associated.
- Independent t-test of tenure between churners and non-churners: $t = -31.5796$, $p = 7.9991 \times 10^{-205}$. Since the p value is less than 0.05 the difference in tenure between customers who churn and those who stay is statistically significant.
- Correlation analysis: The correlation between tenure and TotalCharges is approximately 0.83, between tenure and MonthlyCharges is 0.25, and between MonthlyCharges and TotalCharges is 0.65. These results indicate a strong positive relationship between tenure and TotalCharges.

Multivariate analysis. The pairplot and the correlation heatmap show how tenure, monthly charges and total charges vary together.

Interpretation of patterns. The analysis indicates that churn is associated with shorter customer tenure and month-to-month contracts. Customers who churn also tend to have higher monthly charges. Fiber optic service and electronic-check payment are positively associated with churn. The chi-square test confirms a statistically significant association between contract type and churn, while the t-test indicates a significant difference in tenure between churners and non-churners.

6.2 Feature analysis

Figure F1 shows each feature's correlation with churn, and Figure F2 shows mutual information scores. The top positive features include InternetService_Fiber optic, PaymentMethod_Electronic check, PaperlessBilling and MonthlyCharges. The top negative features include tenure, Contract_Two year, InternetService_No and TotalCharges. The mutual information ranking agrees with the correlation ranking (tenure, Contract_Two year, fiber optic, electronic check, MonthlyCharges). Tenure and TotalCharges are highly correlated (0.83), so both were kept because an MLP tolerates correlated inputs.

Figure F1

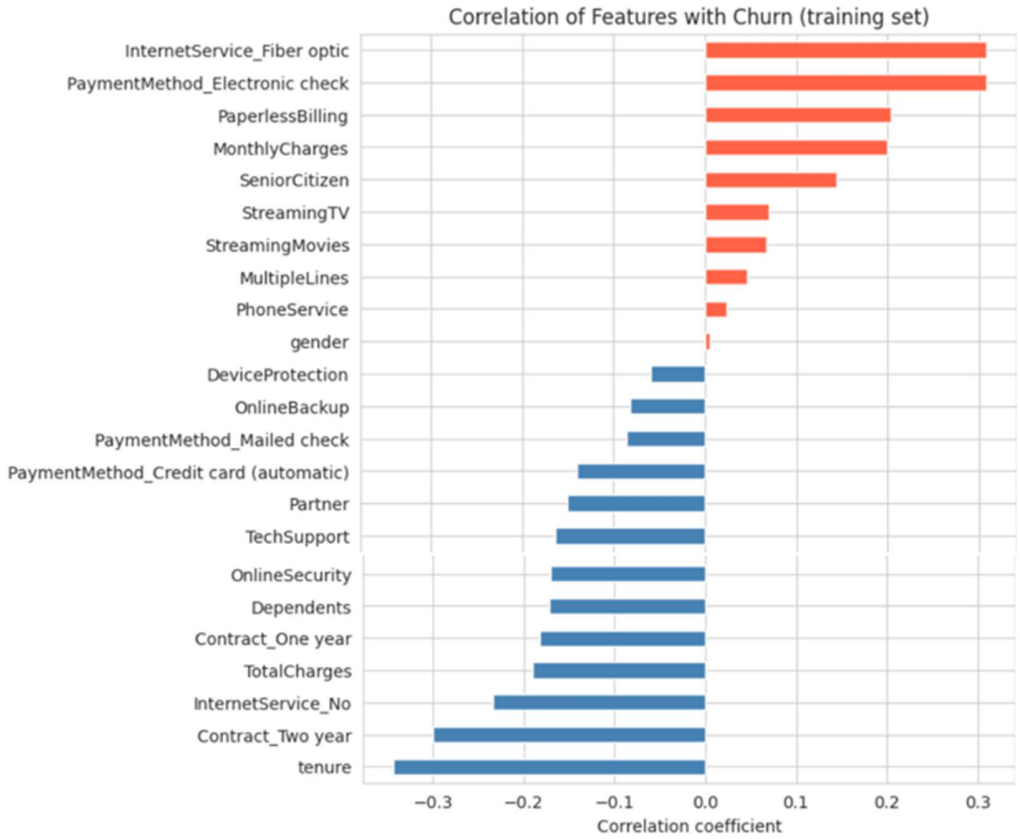
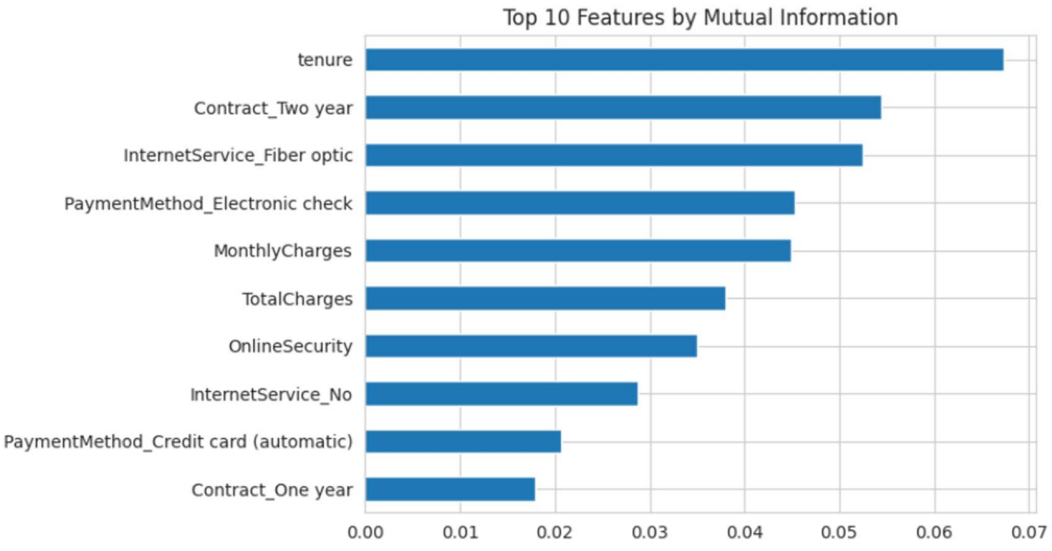


Figure F2



7. Deep Learning Model

7.1 Model choice. The data is tabular, so a Multi-Layer Perceptron (MLP) is appropriate [2]. CNNs are designed for images and RNN/LSTM models for sequences, which this data is not.

7.2 Architecture.

Layer	Details
Input	23 features
Dense 1	128 units, ReLU
Dropout 1	0.4
Dense 2	64 units, ReLU
Dropout 2	0.4
Output	1 neuron, Sigmoid (churn probability)

7.3 Training parameters.

Parameter	Value
Loss function	Binary cross-entropy
Optimizer	Adam
Batch size	32
Maximum epochs	100
Early stopping	Monitor val_loss, patience 10, restore best weights
Class imbalance	Balanced class weights
Metrics	Accuracy, AUC
Random seed	42

7.4 Hyperparameter tuning. Four configurations were compared using validation AUC.

Units	Dropout	Learning rate	Best validation AUC
32	0.2	0.001	0.8468
64	0.3	0.001	0.8450
64	0.3	0.0005	0.8427
128	0.4	0.001	0.8470

The configuration with 128 units, 0.4 dropout, and a learning rate of 0.001 achieved the highest validation AUC of 0.8470 and was selected as the final model. Selection was based on the validation set only, while the test set was kept untouched until the final evaluation.

8. Result Visualization and Interpretation

Test-set metrics (from classification_report): Accuracy is 0.75 ; Precision is 0.51 ; Recall is 0.80 (churn class); F1-score is 0.63 ; AUC 0.839.

Figure R1: Loss vs Epoch. Shows training and validation loss at each epoch. Training loss decreased from approximately 0.558 to 0.481, while validation loss fluctuated and ended near 0.510. The lowest validation loss was approximately 0.508 at epoch 6. The difference between training and validation loss indicates some overfitting.

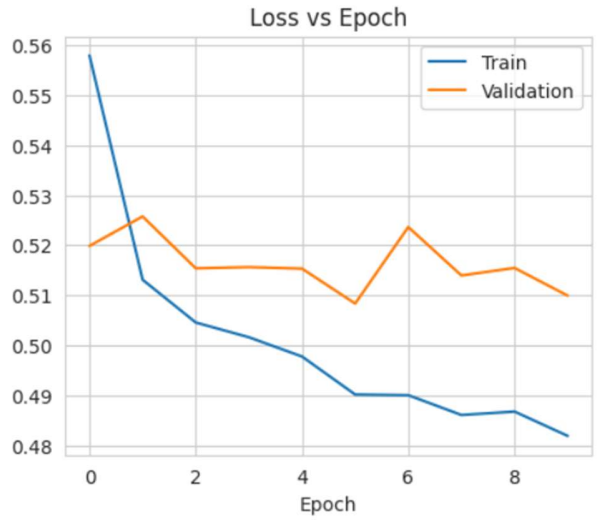


Figure R1b: Accuracy vs Epoch

Validation accuracy fluctuated around 72–73%, while training accuracy increased to approximately 75.5% by the final epoch.

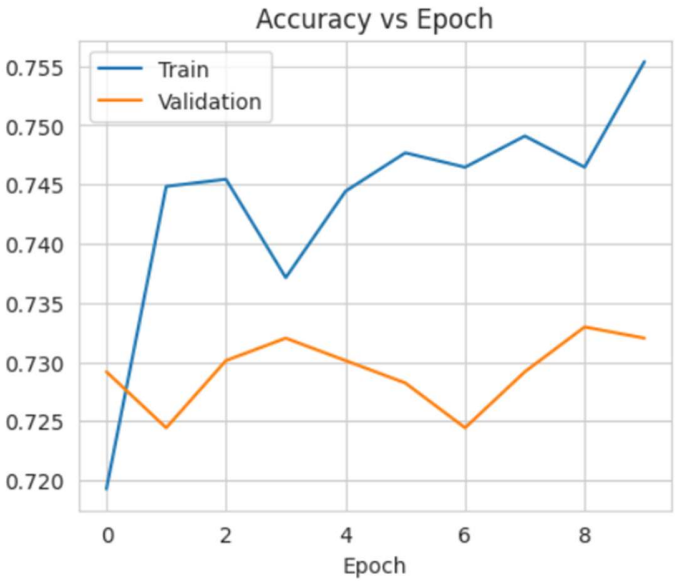


Figure R2: Confusion matrix. TN = 562, FP = 214, FN = 55, TP = 226. The model correctly identified 226 churners and missed 55 churners. Since missing a churner can lead to customer loss, the churn recall of 80% is important. The model identified 80% of actual churners, although it also produced 214 false positives.

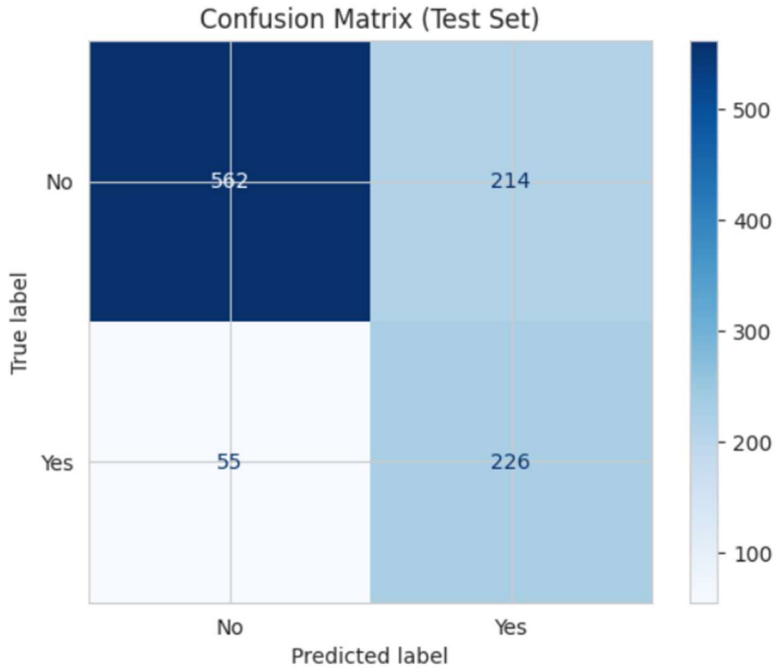


Figure R3: ROC curve and AUC. AUC = 0.839. The curve sits well above the diagonal, indicating good separation between churners and non-churners.

In simple words, the AUC of 0.839 means the model has a good ability to distinguish a randomly selected churner from a randomly selected non-churner.

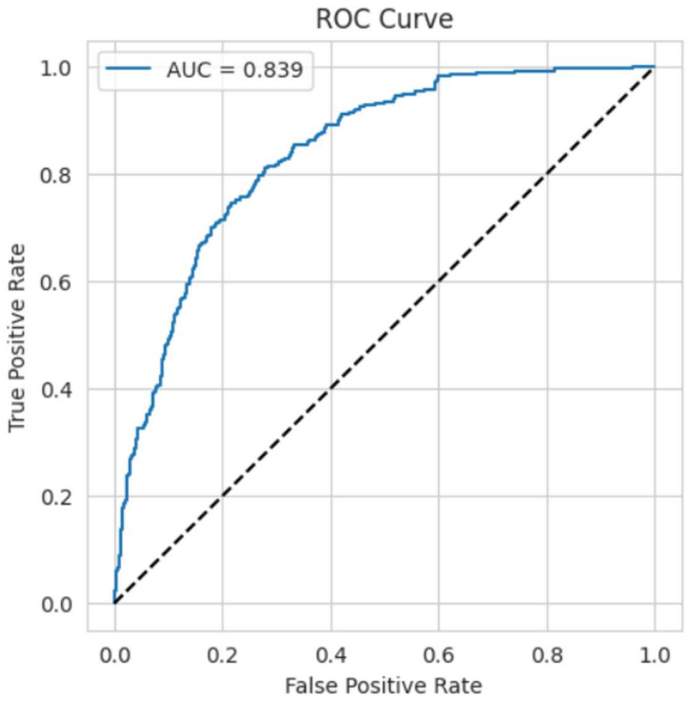
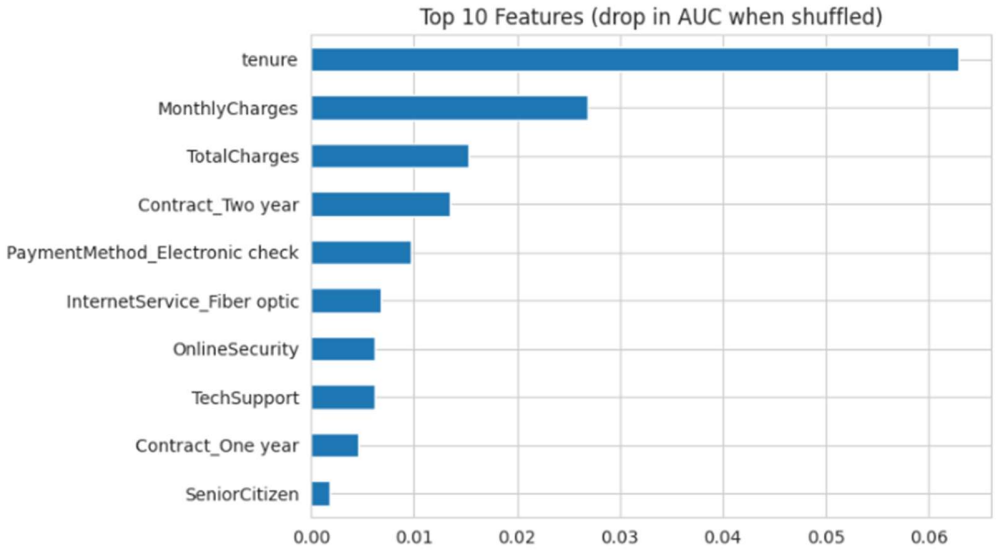


Figure R4: Permutation feature importance. The top features were tenure, MonthlyCharges, TotalCharges, Contract_Two year and PaymentMethod_Electronic check. This agrees with the EDA and with Figures F1 and F2: tenure, contract type, electronic check payment and fiber optic internet rank near the top in all of them, which strengthens confidence in the findings.



9. Conclusion and Future Scope

9.1 What was learned. The dataset showed that customer churn is influenced by factors such as tenure, contract type, and monthly charges. New customers and customers with month-to-month contracts are

important groups to monitor. The MLP model helped predict customer churn and identify customers who may leave the company.

9.2 Key findings.

1. About 26.5% of customers churned (class imbalance).
2. Tenure finding: Customers with shorter tenure were more likely to churn, while customers who stayed longer were less likely to leave.
3. Contract finding: Customers with month-to-month contracts were more likely to churn than customers with one-year or two-year contracts.
4. Charges finding: MonthlyCharges and TotalCharges were among the important features influencing the model's predictions.
5. The MLP achieved 75% accuracy and 0.839 AUC on unseen test data.

9.3 Model performance. The MLP model achieved 75% accuracy and an AUC of 0.839, indicating good ability to distinguish between churners and non-churners. Churn precision was 51% and recall was 80%, meaning the model identified most actual churners but also incorrectly flagged some customers who would not churn.

9.4 Challenges faced.

- Class imbalance (handled with class weights).
- Blank values in TotalCharges stored as text.
- Many categorical features requiring encoding.
- Avoiding data leakage (scaler fitted only on training data).
- Interpreting model performance using the classification report, confusion matrix, ROC curve, and AUC score.

9.5 Practical recommendations for the business.

- Offer discounts or incentives to move month-to-month customers onto one- or two-year contracts.
- Give extra attention and onboarding support to customers in their first months.
- Promote add-ons such as tech support and online security which are associated with lower churn in this data (correlation about -0.16 and -0.17).

9.6 Limitations.

The dataset is a single snapshot with no time dimension, and it does not contain data on customer complaints or competitor offers.

9.7 Future scope.

- Handle imbalance with SMOTE or focal loss.
- Compare the MLP with Logistic Regression, Random Forest and XGBoost.
- Use SHAP or LIME for explainable predictions.
- Perform more extensive tuning with Keras Tuner or Optuna and cross-validation.
- Add features such as customer complaints, support calls and usage trends.
- Deploy the model as a Streamlit or Flask app for real-time churn scoring.

10. References (IEEE style)

[1] IBM, "Telco Customer Churn," Kaggle dataset (blastchar), https://www.kaggle.com/datasets/blastchar/telco-customer-churn [1]. Accessed: 14 Oct. 2026.

[2] F. Chollet, Deep Learning with Python, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2021.

[3] B. Fry, Visualizing Data: Exploring and Explaining Data with the Processing Environment. Sebastopol, CA, USA: O'Reilly Media, 2007.

[4] M. L. Waskom, "seaborn: statistical data visualization," Journal of Open Source Software, vol. 6, no. 60, p. 3021, 2021.

[5] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. Available: https://www.tensorflow.org

[6] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

11. Appendix (Code Section)

GitHub link : https://github.com/SrisivaranjaniM/telco-churn-eda-project

IMPORTS AND DATA LOADING

```
import os

import numpy as np, pandas as pd

import matplotlib.pyplot as plt, seaborn as sns

from scipy import stats

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import (confusion_matrix, ConfusionMatrixDisplay,

                             roc_curve, roc_auc_score, classification_report)

from sklearn.utils.class_weight import compute_class_weight

import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers

os.makedirs("images", exist_ok=True)

sns.set_style("whitegrid")

tf.random.set_seed(42); np.random.seed(42)

df = pd.read_csv("/content/WA_Fn-UseC_-Telco-Customer-Churn.csv")   # ACQUIRE
```

```
print(df.shape)
df.head()
```

UNDERSTANDING OF DATA (EDA)

```
df.info()
print(df.describe())
print("Duplicates:", df.duplicated().sum())
print(df.isnull().sum())
print(df["Churn"].value_counts(normalize=True))
```

CLEANING AND PREPROCESSING (PARSE AND FILTER)

```
# TotalCharges is text because ~11 rows are blank (customers with tenure = 0)
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
print("Missing TotalCharges:", df["TotalCharges"].isnull().sum())
df["TotalCharges"] = df["TotalCharges"].fillna(df["TotalCharges"].median())
df = df.drop_duplicates()
df = df.drop(columns=["customerID"])          # ID has no predictive value
# Merge redundant categories
df = df.replace({"No internet service": "No", "No phone service": "No"})
# Outlier check (IQR method)
for col in ["tenure", "MonthlyCharges", "TotalCharges"]:
    q1, q3 = df[col].quantile([.25, .75]); iqr = q3 - q1
    n = ((df[col] < q1 - 1.5*iqr) | (df[col] > q3 + 1.5*iqr)).sum()
    print(f"{col}: {n} outliers")
# Decide and justify in the report (they are likely genuine values, so keep them)
df_clean = df.copy()   # keep readable version for visualizations
```

VISUALIZATIONS (MINE ,REPRESENT ,REFINE)

```
# V1: Target distribution
plt.figure(figsize=(5,4))
sns.countplot(data=df_clean, x="Churn", palette="Set2")
plt.title("Customer Churn Distribution"); plt.xlabel("Churn"); plt.ylabel("Customers")
```

```
plt.savefig("images/v1_churn_count.png", dpi=150, bbox_inches="tight"); plt.show()
# V2: Tenure histogram by churn
plt.figure(figsize=(7,4))
sns.histplot(data=df_clean, x="tenure", hue="Churn", kde=True, bins=30)
plt.title("Tenure Distribution by Churn"); plt.xlabel("Tenure (months)")
plt.savefig("images/v2_tenure_hist.png", dpi=150, bbox_inches="tight"); plt.show()
# V3: Boxplot of monthly charges
plt.figure(figsize=(6,4))
sns.boxplot(data=df_clean, x="Churn", y="MonthlyCharges", palette="Set2")
plt.title("Monthly Charges vs Churn")
plt.savefig("images/v3_box_monthly.png", dpi=150, bbox_inches="tight"); plt.show()
# V4: Contract type vs churn
plt.figure(figsize=(7,4))
sns.countplot(data=df_clean, x="Contract", hue="Churn", palette="Set2")
plt.title("Churn by Contract Type")
plt.savefig("images/v4_contract.png", dpi=150, bbox_inches="tight"); plt.show()
# V5: Scatterplot
plt.figure(figsize=(7,5))
sns.scatterplot(data=df_clean, x="tenure", y="MonthlyCharges", hue="Churn", alpha=.5)
plt.title("Tenure vs Monthly Charges")
plt.savefig("images/v5_scatter.png", dpi=150, bbox_inches="tight"); plt.show()
# V6: Correlation heatmap (numeric columns)
plt.figure(figsize=(5,4))
sns.heatmap(df_clean[["tenure","MonthlyCharges","TotalCharges","SeniorCitizen"]].corr(),
        annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.savefig("images/v6_heatmap.png", dpi=150, bbox_inches="tight"); plt.show()

# V7: Pairplot
sns.pairplot(df_clean[["tenure","MonthlyCharges","TotalCharges","Churn"]], hue="Churn", corner=True)
```

```
plt.savefig("images/v7_pairplot.png", dpi=150, bbox_inches="tight"); plt.show()
```

STATISTICAL AND DISTRIBUTION ANALYSIS

```
# Continuous: MonthlyCharges, with skewness/kurtosis and a normal fit
x = df_clean["MonthlyCharges"]
print("Mean", x.mean(), "Median", x.median(), "Std", x.std(),
      "Skew", stats.skew(x), "Kurtosis", stats.kurtosis(x))
fig, ax = plt.subplots(1, 2, figsize=(11,4))
sns.histplot(x, stat="density", bins=30, ax=ax[0])
xs = np.linspace(x.min(), x.max(), 200)
ax[0].plot(xs, stats.norm.pdf(xs, x.mean(), x.std()), "r")
ax[0].set_title("MonthlyCharges with Normal Fit")
stats.probplot(x, dist="norm", plot=ax[1]); ax[1].set_title("Q-Q Plot")
plt.savefig("images/dist_continuous.png", dpi=150, bbox_inches="tight"); plt.show()
print(stats.shapiro(x.sample(500, random_state=1)))   # normality test
# Discrete: number of add-on services per customer vs Binomial
services =
["OnlineSecurity","OnlineBackup","DeviceProtection","TechSupport","StreamingTV","StreamingMovie
s"]
n_serv = (df_clean[services] == "Yes").sum(axis=1)
p = n_serv.mean() / len(services)
vals = np.arange(0, len(services)+1)
plt.figure(figsize=(7,4))
plt.bar(vals, n_serv.value_counts(normalize=True).reindex(vals, fill_value=0), alpha=.6,
label="Observed")
plt.plot(vals, stats.binom.pmf(vals, len(services), p), "ro-", label="Binomial fit")
plt.title("Add-on Services per Customer: Observed vs Binomial")
plt.xlabel("Number of services"); plt.legend()
plt.savefig("images/dist_discrete.png", dpi=150, bbox_inches="tight"); plt.show()
# Bivariate: chi-square test (categorical) and t-test (numeric)
ct = pd.crosstab(df_clean["Contract"], df_clean["Churn"])
print("Chi-square p-value:", stats.chi2_contingency(ct)[1])
```

```
print(stats.ttest_ind(df_clean[df_clean.Churn=="Yes"]["tenure"],
                      df_clean[df_clean.Churn=="No"]["tenure"]))
```

ENCODING , SPLITTING AND SCALING

```
df_m = df_clean.copy()
yes_no = ["Partner","Dependents","PhoneService","MultipleLines","OnlineSecurity","OnlineBackup",
        "DeviceProtection","TechSupport","StreamingTV","StreamingMovies","PaperlessBilling","Churn"]
for c in yes_no:
    df_m[c] = (df_m[c] == "Yes").astype(int)
df_m["gender"] = (df_m["gender"] == "Male").astype(int)
df_m = pd.get_dummies(df_m, columns=["InternetService","Contract","PaymentMethod"],
                      drop_first=True, dtype=int)
X = df_m.drop(columns="Churn"); y = df_m["Churn"]
# 70 / 15 / 15 stratified split
X_tr, X_tmp, y_tr, y_tmp = train_test_split(X, y, test_size=0.30, stratify=y, random_state=42)
X_val, X_te, y_val, y_te = train_test_split(X_tmp, y_tmp, test_size=0.50, stratify=y_tmp,
random_state=42)
print(X_tr.shape, X_val.shape, X_te.shape)
# Scale numeric columns (fit on TRAIN only, to avoid data leakage)
num_cols = ["tenure","MonthlyCharges","TotalCharges"]
sc = StandardScaler().fit(X_tr[num_cols])
for d in (X_tr, X_val, X_te):
    d[num_cols] = sc.transform(d[num_cols])
```

DEEP LEARNING MODEL (MLP) WITH BASIC HYPERPARAMETER TUNING

```
cw = compute_class_weight("balanced", classes=np.array([0,1]), y=y_tr)
class_weight = {0: cw[0], 1: cw[1]}          # handles class imbalance
def build(units, dropout, lr):
    m = keras.Sequential([
        layers.Input(shape=(X_tr.shape[1],)),
        layers.Dense(units, activation="relu"), layers.Dropout(dropout),
        layers.Dense(units//2, activation="relu"), layers.Dropout(dropout),
```

```
        layers.Dense(1, activation="sigmoid")])
    m.compile(optimizer=keras.optimizers.Adam(lr), loss="binary_crossentropy",
              metrics=["accuracy", keras.metrics.AUC(name="auc")])
    return m

es = keras.callbacks.EarlyStopping(monitor="val_loss", patience=10, restore_best_weights=True)
grid = [(32,0.2,1e-3), (64,0.3,1e-3), (64,0.3,5e-4), (128,0.4,1e-3)]
best = None
for units, dr, lr in grid:
    m = build(units, dr, lr)
    h = m.fit(X_tr, y_tr, validation_data=(X_val, y_val), epochs=100, batch_size=32,
              class_weight=class_weight, callbacks=[es], verbose=0)
    v_auc = max(h.history["val_auc"])
    print(f"units={units}, dropout={dr}, lr={lr} -> best val AUC = {v_auc:.4f}")
    if best is None or v_auc > best[0]:
        best = (v_auc, m, h, (units, dr, lr))
_, model, history, best_params = best
print("Best hyperparameters:", best_params)
model.summary()
```

EVALUATION VISUALIZATIONS

```
# Loss and accuracy vs epoch
fig, ax = plt.subplots(1, 2, figsize=(11,4))
ax[0].plot(history.history["loss"], label="Train"); ax[0].plot(history.history["val_loss"],
label="Validation")
ax[0].set_title("Loss vs Epoch"); ax[0].set_xlabel("Epoch"); ax[0].legend()
ax[1].plot(history.history["accuracy"], label="Train"); ax[1].plot(history.history["val_accuracy"],
label="Validation")
ax[1].set_title("Accuracy vs Epoch"); ax[1].set_xlabel("Epoch"); ax[1].legend()
plt.savefig("images/r1_curves.png", dpi=150, bbox_inches="tight"); plt.show()
# Test predictions
prob = model.predict(X_te).ravel(); pred = (prob >= 0.5).astype(int)
```

```
print(classification_report(y_te, pred, target_names=["No churn","Churn"]))

# Confusion matrix

ConfusionMatrixDisplay(confusion_matrix(y_te, pred),
display_labels=["No","Yes"]).plot(cmap="Blues")

plt.title("Confusion Matrix (Test Set)")

plt.savefig("images/r2_confusion.png", dpi=150, bbox_inches="tight"); plt.show()

# ROC curve and AUC

fpr, tpr, _ = roc_curve(y_te, prob); auc = roc_auc_score(y_te, prob)

plt.figure(figsize=(5,5)); plt.plot(fpr, tpr, label=f"AUC = {auc:.3f}"); plt.plot([0,1],[0,1],"k--")

plt.xlabel("False Positive Rate"); plt.ylabel("True Positive Rate"); plt.title("ROC Curve"); plt.legend()

plt.savefig("images/r3_roc.png", dpi=150, bbox_inches="tight"); plt.show()
```

FEATURE IMPORTANCE

```
base = roc_auc_score(y_te, model.predict(X_te, verbose=0).ravel())

imp = {}

for col in X_te.columns:

    Xp = X_te.copy(); Xp[col] = np.random.permutation(Xp[col].values)

    imp[col] = base - roc_auc_score(y_te, model.predict(Xp, verbose=0).ravel())

imp = pd.Series(imp).sort_values(ascending=False).head(10)

imp[::-1].plot(kind="barh", figsize=(7,5), title="Top 10 Features (drop in AUC when shuffled)")

plt.savefig("images/r4_importance.png", dpi=150, bbox_inches="tight"); plt.show()
```

FEATURE ANALYSIS

```
from sklearn.feature_selection import mutual_info_classif

# 1) Correlation of every (encoded) feature with Churn
corr = pd.concat([X_tr, y_tr], axis=1).corr()["Churn"].drop("Churn").sort_values()
plt.figure(figsize=(7,8))
corr.plot(kind="barh", color=["tomato" if v > 0 else "steelblue" for v in corr])
plt.title("Correlation of Features with Churn (training set)")
plt.xlabel("Correlation coefficient")
plt.savefig("images/f1_corr_target.png", dpi=150, bbox_inches="tight"); plt.show()

# 2) Mutual information (captures non-linear relationships too)
mi = pd.Series(mutual_info_classif(X_tr, y_tr, random_state=42),
               index=X_tr.columns).sort_values(ascending=False)
mi.head(10)[::-1].plot(kind="barh", figsize=(7,5), title="Top 10 Features by Mutual Information")
```

```
plt.savefig("images/f2_mutual_info.png", dpi=150, bbox_inches="tight"); plt.show()

# 3) Multicollinearity check: tenure and TotalCharges are expected to be highly correlated
print(X_tr[["tenure","MonthlyCharges","TotalCharges"]].corr())
# Summary statistics for numeric columns
df_clean[["tenure","MonthlyCharges","TotalCharges"]].agg(["mean","median","std","skew","kurt"])
```